UTM

UNIVERSITI TEKNOLOGI MALAYSIA

UNIVERSITI PENYELIDIKAN

SECJ2013

DATA STRUCTURE AND ALGORITHM

SECTION 02

ASSIGNMENT 1

LECTURER:

DR. ZURAINI BINTI ALI SHAH

GROUP 3 MEMBER:

LIM YU HAN (A23CS0241)

EVELYN GOH YUAN QI (A23CS0222)

ANIS SAFIYYA BINTI JANAI (A23CS0049)

PRAVINRAJ A/L SIVABATHI (A23CS0171)

Table of Contents

1. Sorting Algorithms

1.1 Bubble Sort

```cpp
// bubble sort

#include <iostream>
using namespace std;

void BubbleSort (int data[], int listSize)
{
    int pass, tempValue;
    int comparisonCount=0;
    int swapCount=0;

    for (pass = 1; pass < listSize; pass++)
    { // moves the largest element to the end of the array
        for (int x = 0; x < listSize - pass; x++)
        { // compare adjacent elements
        comparisonCount++;
            if ( data[x] > data[x+1] )
            { // swap elements
                tempValue = data[x];
                data[x] = data[x+1];
                data[x+1] = tempValue;

                swapCount++;
            }
        }
    }

    cout << "\nTotal number of comparisons: " << comparisonCount << endl;
    cout << "Total number of swaps: " << swapCount << endl;

} // end Bubble Sort


void printArray(int data[], int listSize)
{
    for(int i = 0; i < listSize; i++)
    {
        cout << data[i] << " ";
    }
    cout << endl;
}

int main()
{
    int studentMarks[] = {75,95,60,88,70};
    int list=5;

    cout << "Original array: ";
    printArray(studentMarks, list);

    BubbleSort(studentMarks, list);

    cout << "\nBubble Sorted array in ascending order: ";
    printArray(studentMarks, list);

    return 0;
}
```

Output:

```
Original array: 75 95 60 88 70

Total number of comparisons: 10
Total number of swaps: 6

Bubble Sorted array in ascending order: 60 70 75 88 95
```

1.2 Improved Bubble Sort

```
#include <iostream>
using namespace std;

void improvedBubbleSort (int data[], int n)
{       int pass,temp;
        int comparisonCount=0;
        int swapCount=0;
         bool sorted = false; // false when swaps occur
         for (int pass = 1; (pass < n) && !sorted; ++pass)
         {     sorted = true; // assume sorted
                for (int x = 0; x < n-pass; ++x)
                {
                    comparisonCount++;
                    if (data[x] > data[x+1])
                        {     // exchange items
                            temp = data[x];
                            data[x] = data[x+1];
                            data[x+1] = temp;
                            sorted = false; // signal exchange
                            swapCount++;
                         } // end if
                } // end for
        }//outer for
    cout<<"\nTotal number of comparisons: "<<comparisonCount<<endl;
    cout<<"Total number of swap: "<<swapCount<<endl;
} //end

void printArray(int data[], int listSize)
{
    for (int i=0; i<listSize; i++)
    {
        cout<<data[i]<<" ";
    }
    cout<<endl;
}


int main()
{
    int studentMarks[]={75,95,60,88,70};   //unsorted data

    int list = 5;
    cout<<"Original array: ";
    printArray(studentMarks,list);

    improvedBubbleSort(studentMarks,list);

    cout<<"\nImproved bubble sorted array in ascending order: ";
    printArray(studentMarks,list);

    return 0;
}
```

Output:

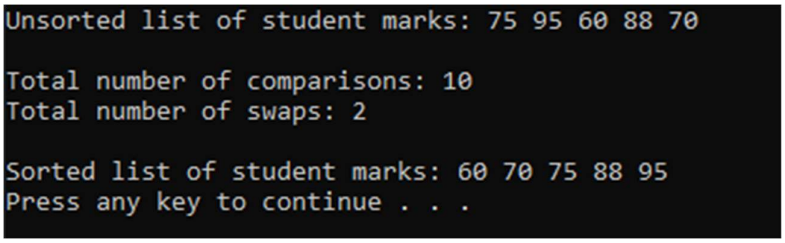
```
Original array: 75 95 60 88 70

Total number of comparisons: 10
Total number of swap: 6

Improved bubble sorted array in ascending order: 60 70 75 88 95

--------------------------------
Process exited after 0.7206 seconds with return value 0
Press any key to continue . . .
```

1.3 Selection Sort

```
void swap(int& x, int& y)
{
    int temp = x;
    x = y;
    y = temp;
} // end swap

void printArray(int Data[], int arraySize)
{
    for (int i = 0; i < arraySize; i++)
    {
        cout<< Data[i] << " ";
    }
    cout<< endl;
}

int main()
{
    int studentMarks[] = {75, 95, 60, 88, 70};
    int arraySize = 5;      // 5 student marks

    cout<<"Unsorted list of student marks: ";
    printArray(studentMarks, arraySize);

    selectionSort(studentMarks, arraySize);

    cout<<"\n\nSorted list of student marks: ";
    printArray(studentMarks,arraySize);

    system("pause");
    return 0;
}
```

```
#include <iostream>
using namespace std;

void selectionSort(int Data[], int n)
{
    int comparisonNum = 0;
    int swapNum = 0;

    for (int last = n-1; last >= 1; --last)
    {
        // select largest item in the Array
        int largestIndex = 0;

        // largest item is assumed start at index 0
        for (int p=1; p <= last; ++p)
        {
            if (Data[p] > Data[largestIndex])
            largestIndex = p;
            ++comparisonNum;
        }

        // swap largest item Data[largestIndex] with Data[last]
        if (Data[largestIndex] != Data[last])
        {
            swap(Data[largestIndex],Data[last]);
            ++swapNum;
        }
    }
    cout<< "\nTotal number of comparisons: " << comparisonNum;
    cout<< "\nTotal number of swaps: " << swapNum;

} // end selectionSort
```

Output:


Unsorted list of student marks: 75 95 60 88 70
Total number of comparisons: 10
Total number of swaps: 2
Sorted list of student marks: 60 70 75 88 95
Press any key to continue . . .

1.4 Insertion Sort

```cpp
#include <iostream>
using namespace std;

void insertionSort(int ary[], int size, int &comp, int &swap) {
    for (int i = 1; i < size; i++) {
        int key = ary[i];
        int j = i - 1;

        while (j >= 0 && ary[j] > key) {
            comp++;
            ary[j + 1] = ary[j];
            swap++;
            j--;
        }

        ary[j + 1] = key;

        if (j >= 0) {
            comp++;
        }
    }
}

void printArray(int ary[], int size) {
    for (int i = 0; i < size; i++) {
        cout << ary[i] << " ";
    }
    cout << endl;
}
```

```cpp
int main() {

    int mark[] = {75, 95, 60, 88, 70};
    int size = sizeof(mark) / sizeof(mark[0]);

    int comp = 0;
    int swap = 0;

    cout << "Original marks: ";
    printArray(mark, size);
    insertionSort(mark, size, comp, swap);

    cout << "Sorted marks: ";
    printArray(mark, size);

    cout << "Total comparisons: " << comp << endl;
    cout << "Total swaps: " << swap << endl;

    return 0;
}
```

Output :

```
Original marks: 75 95 60 88 70
Sorted marks: 60 70 75 88 95
Total comparisons: 9
Total swaps: 6

--------------------------------
Process exited after 12 seconds with return value 0
Press any key to continue . . .
```

2. Analysis Report

2.1 Performance Comparison

Bubble Sort

- **Number of comparisons**: The algorithm performed a total of 10 comparison for a list of 5. This matches the expected performance of Bubble Sort, where the number of comparisons for a list of size n is approximately $(n - 1) + (n - 2) + \cdots + 2 + 1 = (n (n - 1)) / 2$.

- **Number of swaps**: The total number of swaps required was 6 for this dataset.

Bubble Sort systematically compares and swaps adjacent elements as needed, ensuring the largest unsorted element "bubbles" to the correct position with each pass.

Improved Bubble Sort

- **Number of Comparisons**: The algorithm performed a total of 10 comparisons for a list of 5 elements. This is identical to the standard Bubble Sort for this dataset, as the array required multiple passes to become fully sorted, and the early termination condition did not come into effect until the final pass.

- **Number of Swaps**: The total number of swaps required was 6, matching the standard Bubble Sort for this dataset. Since every comparison that identified a mismatch led to a swap, the swap count remained unchanged.

Improved Bubble Sort incorporates an optimization by using a flag (sorted) to detect whether any swaps occurred in the current pass. If no swaps are made, the algorithm terminates early, as this indicates the array is already sorted. While this optimization significantly reduces redundant passes for partially sorted or already sorted datasets, it did not show any performance advantage in this test case due to the dataset being completely unsorted at the start.

Selection Sort

- **Number of comparisons**: The algorithm performed a total of 10 comparison for a list of 5 elements. This fits the expected performance of Selection Sort, where the

number of comparisons for a list of size n is approximately (n − 1) + (n – 2) + ⋯ + 2 + 1 = (n (n – 1)) / 2.

- **Number of swaps:** The total number of swaps required was 2 for this dataset.

Selection Sort compares all the elements in the list to find the largest value and swaps the element to the last position of the subarray repetitively in order to sort all the elements.

Insertion Sort

- **Number of comparisons:** The algorithm performed a total of 9 comparisons for a list of 5 elements. This aligns with the expected performance of Insertion Sort, where the number of comparisons depends on the initial order of the elements, typically ranging between n−1 (best case) and $n(n-1)2$ (worst case).
- **Number of swaps:** The total number of swaps required was 6 for this dataset.

Insertion Sort builds the sorted array one element at a time, by comparing and shifting elements as needed to insert the current element into its correct position.

In summary, while Bubble Sort and its improved variant are simple but inefficient for unsorted data, Selection Sort and Insertion Sort offer better performance in terms of swap efficiency and adaptability, respectively. Each algorithm's performance varies based on the initial order and size of the dataset.

2.2 Efficiency of each algorithm

Sorting Algorithm	Efficiency of the algorithm
Bubble Sort	Requires $O(n^2)$ comparisons and $O(n^2)$ swaps, making it poor efficiency and not well-suited for larger datasets but remains effective for small or nearly sorted arrays.
Improve Bubble Sort	Requires $O(n^2)$ comparisons and $O(n^2)$ swaps in the worst case, similar to Bubble Sort, which limits its efficiency for larger datasets. However, its key advantage lies in the best-case scenario, where it requires only O(n) comparisons and O(1) swaps if the array is already sorted. This makes it better suited for small or partially sorted arrays compared to standard Bubble Sort.
Selection Sort	Requires $O(n^2)$ comparisons in all cases including the worst case, best case and average case as it does not depend on the initial arrangement of the data. Thus, partially sorted array will not affect the number of comparisons for selection sort. Furthermore, it performs the fewest swaps, O(n), among the other sorts which makes it efficient for reducing write operations.
Insertion Sort	**Insertion Sort** requires $O(n^2)$ comparisons and $O(n^2)$ shifts in the worst case, similar to Bubble Sort, which limits its efficiency for larger datasets. However, its key advantage lies in the best-case scenario, where it requires only O(n) comparisons and O(1) shifts if the array is already sorted. This makes it better suited for small or partially sorted arrays compared to standard Bubble Sort.

2.3 Summarizing Table

Sorting Algorithm	Comparisons	Swaps
Bubble Sort	10	6
Improved Bubble Sort	10	6
Selection Sort	10	2
Insertion Sort	9	6

3. Reflection

Evelyn Goh Yuan Qi

In my opinion, this assignment provided a comprehensive learning experience that deepened my understanding of sorting algorithms and their practical applications. Implementing my assigned algorithm in C++ challenged me to focus on both correctness and efficiency by tracking the number of comparisons and swaps. It was insightful to compare the performance of the algorithms and analyze how optimizations, such as those in Improved Bubble Sort, could significantly enhance efficiency. Collaborating with my team was a key highlight, as it required clear communication and coordination to consolidate our findings and present a cohesive analysis. Besides, debugging and ensuring accurate data tracking were challenges I overcame by using systematic approaches like intermediate outputs and counter checks. This process not only improved my technical skills but also strengthened my problem-solving abilities. In addition, the comparative analysis of the algorithms helped me appreciate the trade-offs between simplicity and efficiency, preparing me to make informed decisions in real-world scenarios. Overall, this assignment fostered both individual growth and teamwork, leaving me better equipped to approach similar tasks in the future.

Lim Yu Han

The performance and effectiveness of several sorting algorithms, including as Bubble Sort, Selection Sort, and Insertion Sort, were insightfully revealed by this project. We found notable variations in the computing costs and appropriateness for various data types by examining the number of comparisons and swaps needed to sort a dataset. In addition to the technical analysis, this assignment promoted cooperation and teamwork since we divided up the work and discussed the results to guarantee a shared understanding of algorithms. Collaborating enhanced our ability to communicate and solve problems, which allowed me to address obstacles effectively.

Pravinraj

The project gave me valuable insights into the performance of sorting algorithms such as Bubble Sort, Selection Sort, and Insertion Sort. Through analyzing the number of comparisons and swaps done, I came to learn how their efficiencies vary upon dataset size and order. While the logic behind Bubble Sort is rather simple, it is less efficient in handling big data sets compared to its other counterparts. This project highlighted the importance of choosing the appropriate algorithm over different scenarios. Teamwork strengthened our skills of problem-solving and communication, and the practical use of these algorithms has built in me the way to use them amply in real-life situations.

Anis

The project gave me an understanding of how different sorting algorithms work and the importance of algorithmic efficiency while handling data. I learned that different algorithms have different strengths and weaknesses regarding data and context. For instance, some perform well on small datasets, while others do equally well on large ones. The project also taught me how to track algorithm performance, such as counting comparisons and swaps, to evaluate their efficiency. Collaborating with my team allowed me to improve my coding skills and gain new perspectives on problem-solving, preparing me for real-world applications where choosing the right algorithm can make a significant difference in performance.